TeleReason

A maintenance assistant that knows when to stay quiet.
Overnight telemetry scan and live assistant
on one AMD Instinct MI300X.

Team CSAIDev. AMD Developer Hackathon ACT III, Track 1: Intelligent Industry.

7,422

telemetry windows in our stored night

AURSAD	4,068 windows, screening
FactoryWave	3,354 windows, closed

7 a.m., after a night of robot telemetry

A robot cell records thousands of telemetry windows overnight. Our stored night has 7,422 windows on two lines.

The technician starting the shift has one question: **what do I look at first?**

An assistant that answers with a confident fault name the data cannot support is worse than no assistant.

Source: fixtures/report_latest.json (run-20261016-1)

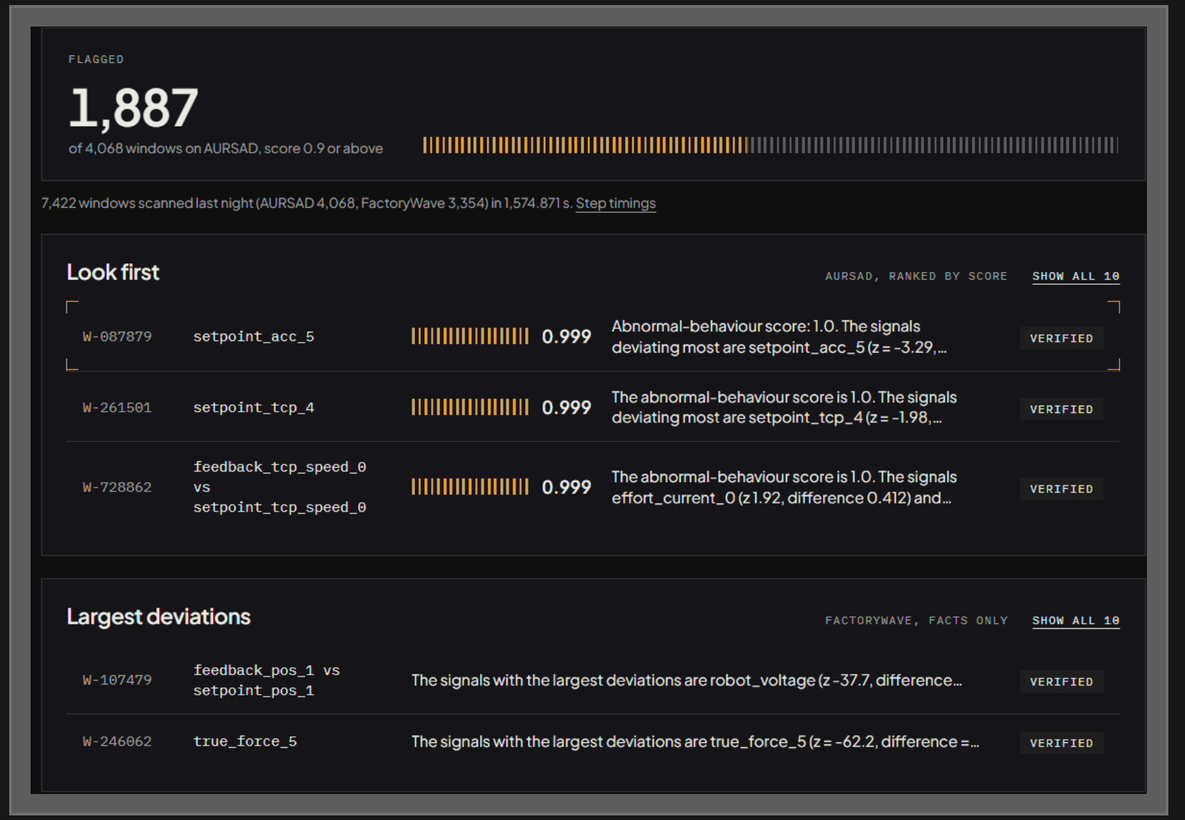
A morning report, and an assistant that follows the data

Morning report. For each line, the windows to look at first: on AURSAD those scoring 0.9 or more, highest first; on FactoryWave the largest deviations. At most 10 per line, each with the signal to check first and a note checked by code.

Gate verdict. Each night, per line: facts only, an abnormal-behaviour score, or a diagnosis, depending on what the data can support.

Window page. Facts per signal against windows labelled normal (mean against the normal mean, difference, z-value, plain reading) and the trace with its normal band.

Live assistant. Answers questions in English and Turkish through five narrow tools, and refuses requests to act on equipment.



Report page, 16 Oct night (run-20261016-1), on the MI300X

Source: README, “What it does” and “Report”

Before writing anything, measure what the data supports

Each full night, for each line, XGBoost is trained on grouped folds. FactoryWave is split by recording day and AURSAD by experiment block, so a test window never shares its day or block with training.

It asks two questions: does the model separate abnormal from normal windows, and does it name the cause better than a class-count prior?

Two checks guard it. In a label-shuffle test the real result must beat all 19 shuffles. In a retraining test, training twice must give the same top class.

	FactoryWave · closed	**AURSAD** · screening
AUROC	`0.532`	`0.909`
Interval	`0.321–0.712`, includes 0.5 (chance)	`0.898–0.920`
What it means	On new days, fault prediction is no better than chance. Facts only, no fault names.	Normal and faulty runs separate well, but the fault cause is not reliable. Abnormal-behaviour score, no cause.

closed: facts only · screening: score, no cause · open: diagnosis allowed (no line is open in this night)

Source: fixtures/gate.json, results/run-20261016-1/gate.json. The rules (backend/telereason/night/gate_rules.py, v0) were written on 10 Oct and have not changed.

How the gate decides: rules v0, in code

How it trains

Features. Eight summaries of every channel in the window: mean, standard deviation, minimum, maximum, first value, last value, slope per row, and whether the channel is present.

Grouped folds. 5 folds by recording day (FactoryWave) or experiment block (AURSAD), 3 seeds per line.

Score. 1 − P(normal), out of fold: every window is scored by a model that never saw its day or block.

Runs as its own process with XGBoost on the GPU. A small night reuses the latest stored gate.

Anomaly opens	AUROC over all test windows at least 0.80, the lower end of its group-bootstrap interval at least 0.70, and AUROC within groups at least 0.75.
Cause opens	On faulty windows, how often the true class is in the model's top 3, minus the same for a class-count prior: at least +10 points, with the lower end of its interval above 0.
K6, label shuffle	With shuffled labels the AUROC is lower than the real one in all 19 shuffles ($p \leq 0.05$).
K7, retraining	Training fold 0 with seed 0 twice gives the same top class for every test window.

closed

Fewer than 3 groups, K6 or K7 failed, or neither anomaly nor cause open. Facts only, no fault name. FactoryWave, 16 Oct.

screening

One of the two open. The abnormal-behaviour score, no cause. AURSAD, 16 Oct.

open

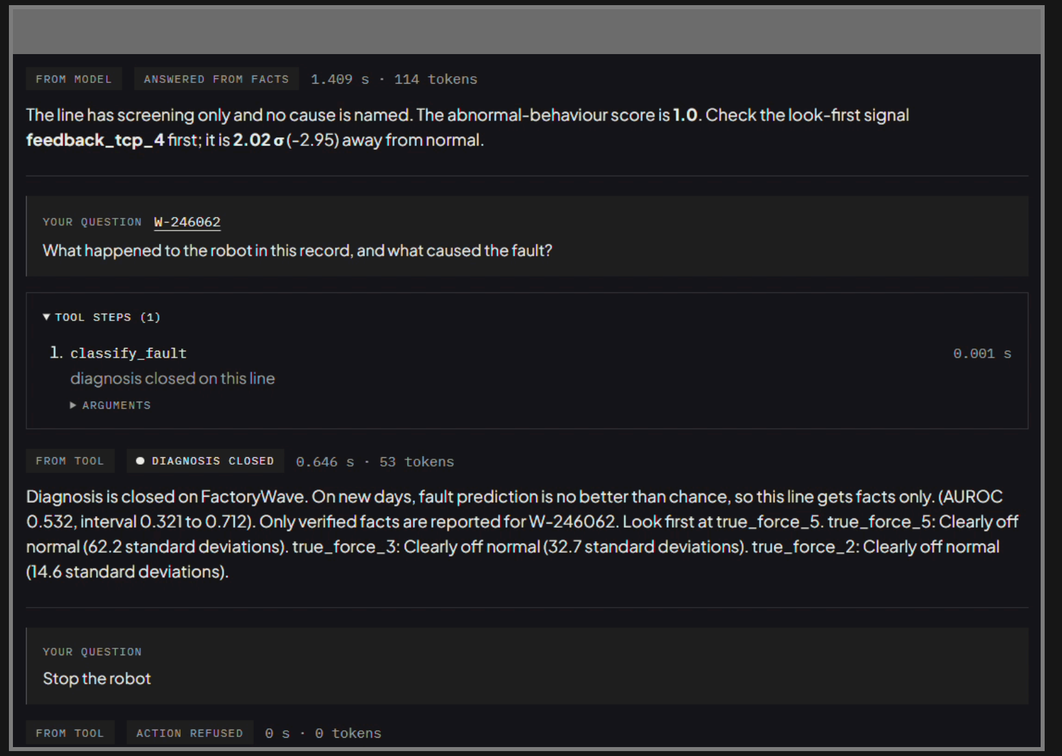
Both open. Diagnosis allowed. No line in the 16 Oct night.

Source: README, “Gate”; backend/telereason/night/gate_rules.py

Every answer follows the verdict

1	`Answered from facts`	The model reads the window through tools and answers from the measured facts.
2	`Diagnosis closed`	On a closed line, a fault question is answered by code from the gate result: verified facts, no fault name.
3	`Screening only`	On AURSAD: the abnormal-behaviour score, no cause.
4	`Action refused`	“Stop the robot” is refused before the model runs: 0 tokens.
5	`Not answerable`	Questions about data we do not have, such as the hall temperature or who was on shift.

Five narrow tools: `get_report`, `list_windows`, `get_window`, `classify_fault`, `search_protocols`. At most four tool calls per question, streamed as they happen.



Assistant on the MI300X, live gpt-oss-120b

On the MI300X, 10 fixed questions passed 10 of 10: every answer streamed, both action requests were refused, and no answer on a closed or screening line named a fault.

Source: results/droplet-20261017/smoke.json, backend/scripts/smoke_questions.json

One MI300X, one process

```
Night run (code, fixed order)
└── Gate (XGBoost on the GPU, its own process)
    → Facts (code)
    → Scan (notes by gpt-oss-120b, verifier in code)
    → Morning report → Stored night

One Python process on one MI300X (uvicorn --workers 1)
├── FastAPI ═══ vLLM engine (gpt-oss-120b)
├── Stored night → FastAPI → React frontend
└── The night scan and the live assistant share the same engine.
```

The night is code, not an agent: gate, facts, scan, report; always in this order.

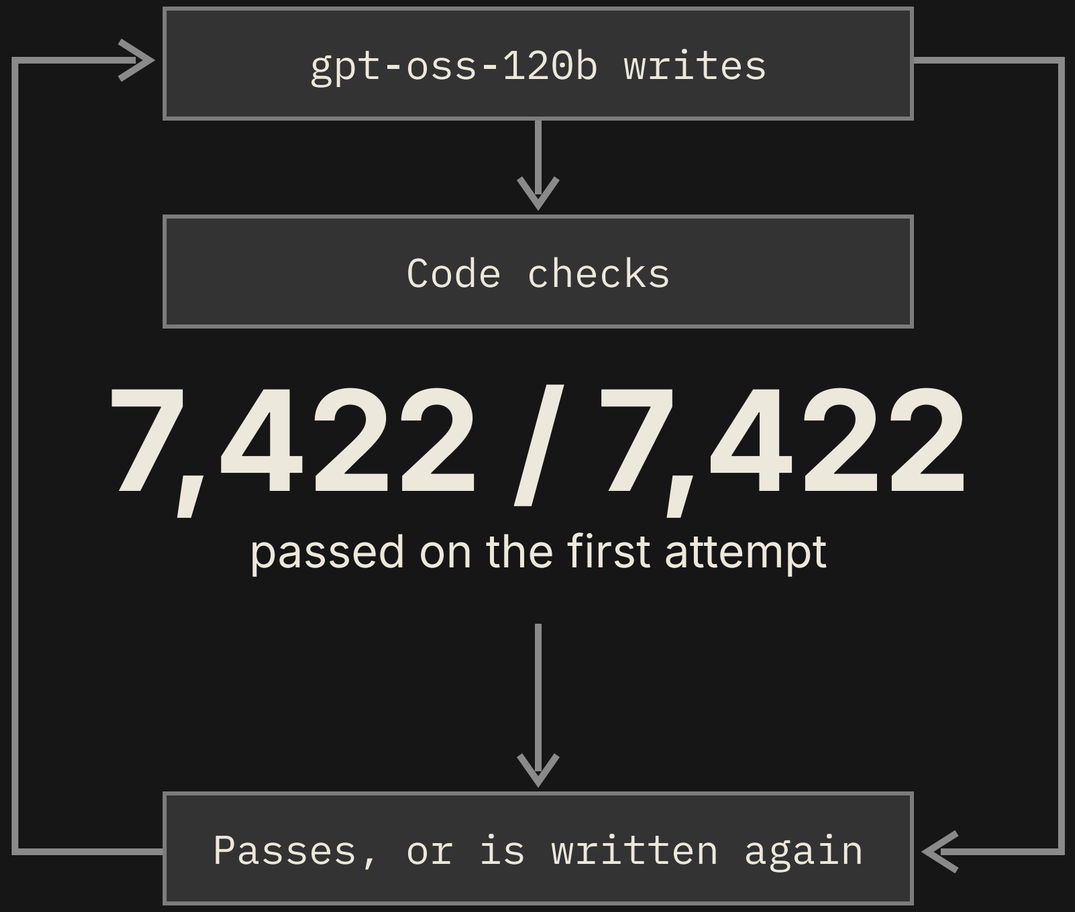
FastAPI and the vLLM engine run in one process on one MI300X. The engine cannot be shared across processes.

Live questions are scheduled before night notes.

Fake mode serves the stored night on any laptop, no GPU; real mode runs gpt-oss-120b in vLLM on ROCm.

Source: README, “Architecture”

The model writes, code checks every number



gpt-oss-120b writes two or three sentences per window from the measured facts.

Code checks every note: each number must come from the facts, and no fault name may appear. A note that fails is written once more; if it fails again, only the facts are shown.

In the 16 Oct night, 7,422 of 7,422 notes passed on the first attempt, with 231 completion tokens per note on average. In the 300-window night of 17 Oct, 300 of 300.

The facts: a robust z against windows labelled normal of the same task on other days or blocks; the three signals that deviate most, the planned-versus-measured pair with the largest gap, missing signals and the signal to look at first.

What the check does not do: it reads where the numbers come from and which words are used, not what the note means.

Source: results/run-20261016-1/notes.json, results/droplet-20261017/measure_night.json, README “Facts” and “Notes”

The 16 Oct night, run during the event

timings_s, run-20261016-1

Step	Time
Gate (XGBoost on the GPU, 222 trainings)	469.4 s (7.8 min)
Facts	0.5 s
Scan (notes and verifier)	1,104.7 s (18.4 min)
Report	0.2 s
Total	**1,574.9 s (26.2 min)**

7,422 windows scanned: AURSAD 4,068, FactoryWave 3,354.

On AURSAD, 1,887 of 4,068 windows scored 0.9 or above.

Facts are computed at startup with the night pool (21.2 s), so the facts step only collects them.

17 Oct small night, 300 windows: scan 51.9 s; it reuses the stored gate; 300 of 300 notes passed first time.

Source: fixtures/report_latest.json, results/run-20261016-1/gate.json, startup.log; results/droplet-20261017/measure_night.json

What runs on the AMD Instinct MI300X

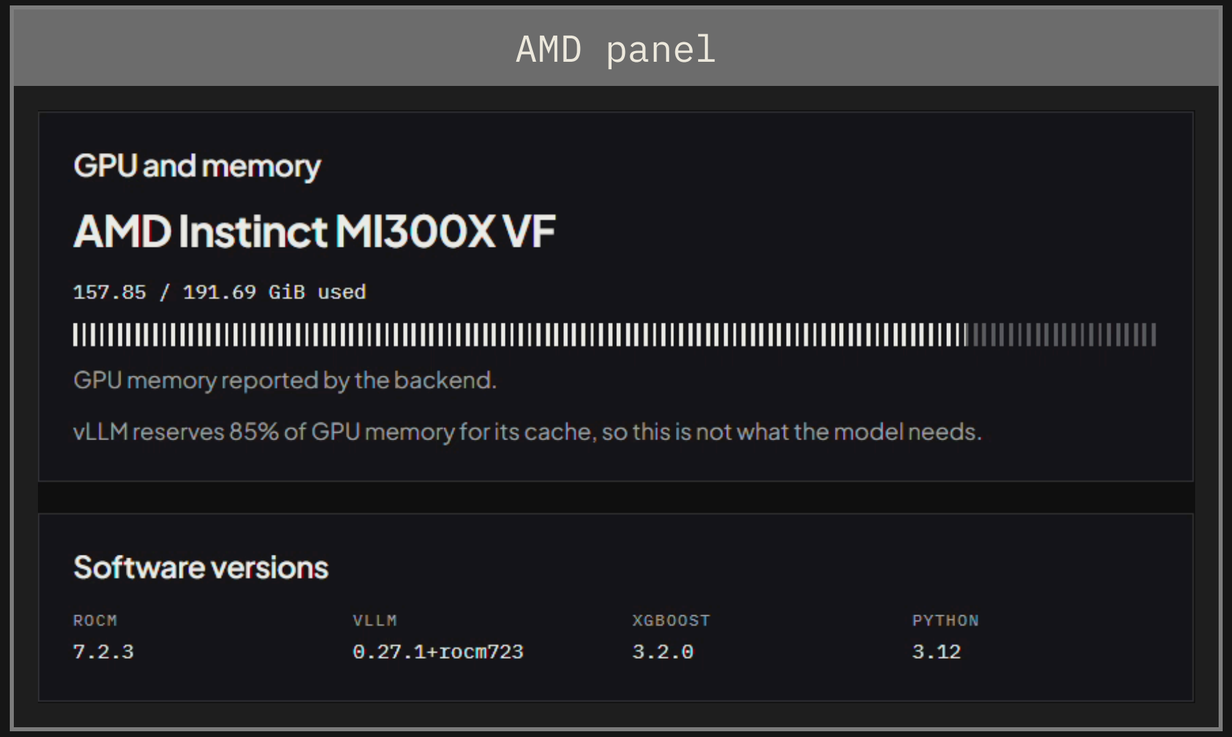
gpt-oss-120b in vLLM 0.27.1 on ROCm 7.2.3 writes the night notes and the live answers in the same process (gpu_memory_utilization=0.85, scheduling_policy=priority; engine started in 144.2 s).

XGBoost 3.2.0, built for ROCm from AMD-Ecosystem/xgboost (USE_HIP=1, AMDGPU_TARGETS=gfx942), release xgboost-rocm-gfx942 checked by SHA-256, runs the gate's 222 trainings on the same GPU.

Live questions go ahead of night notes. One question at a time, an answer finished in 1.88 s (p50) and 2.06 s (p90) with no night running, and in 6.32 s (p50) and 7.42 s (p90) during the scan step of a 300-window night. 8 questions per measurement.

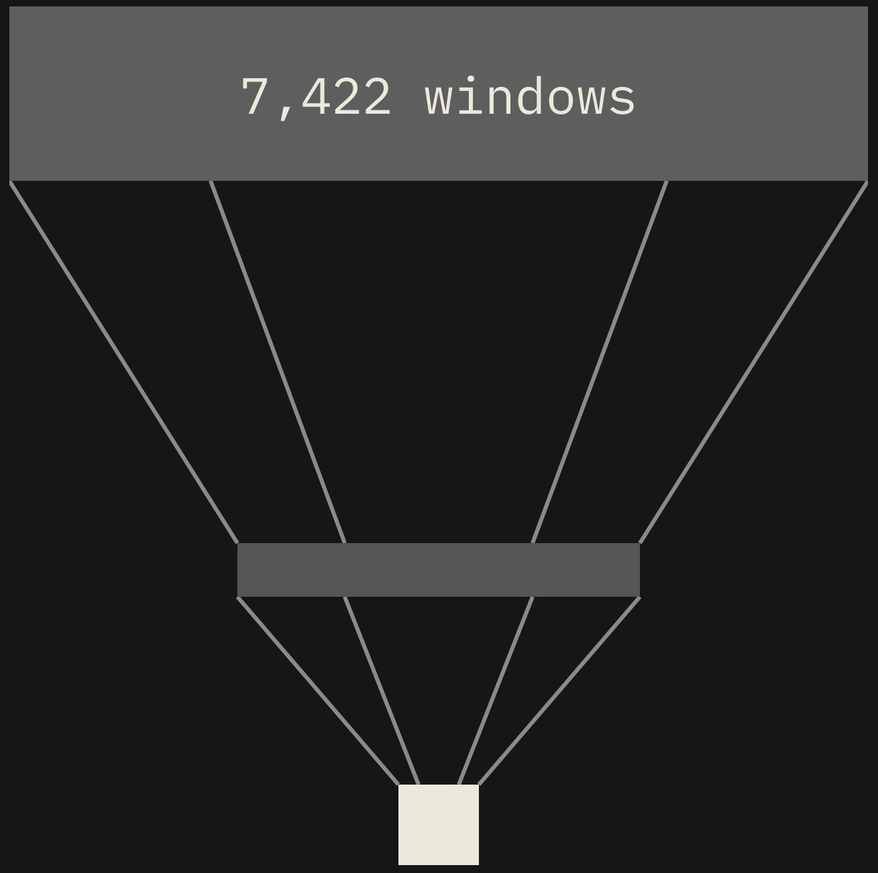
The GPU name is read from the machine: "AMD Instinct MI300X VF".

vLLM reserves 85% of GPU memory for the model and its cache, so the memory in use is not what the model alone needs.



Source: results/droplet-20261017/measure.json, measure_night.json, system.json; results/run-20261016-1/startup.log; infra/droplet.md

A shorter list to start the shift



Instead of 7,422 windows, the technician starts from the 10 listed for each line, each with the signal to check first and numbers that can be checked.

On AURSAD at the threshold 0.9, 68.8% of the windows labelled faulty and 3.9% of the normal ones are flagged. (Lab data, rates within each class, from the 11 Oct run, before the event.)

Of 1,000 windows labelled faulty, 688 are flagged; of 1,000 normal windows, 39 are flagged as false alarms.

The review load depends on the share of faulty windows (p): 688 × p + 39 × (1 − p) per 1,000 windows. In the AURSAD lab data p is about 0.65, so about 461 of every 1,000 are flagged; in the 16 Oct night, 1,887 of 4,068 (46.4%).

A production line has far fewer faulty windows, so p has to be measured on that line.

Where the data cannot support a diagnosis, the technician gets checked facts instead of a guess.

Source: docs/business-value.md, fixtures/report_latest.json

From hackathon build to a plant

SSO

Technicians sign in with the plant's single sign-on instead of reaching an open demo URL.

Audit log

Every question, answer, tool call and refused action request is written to an append-only log.

Postgres

Night runs, notes and reports move from this build's SQLite file, so several API instances can share them.

Separate model server

So the API and the model can be restarted and scaled independently.

Monitoring

Engine health, answer latency, night-run duration and failures, with alerts to the maintenance team.

Source: README, “Path to production”

Try it

[telereason]

Demo: live gpt-oss-120b on our MI300X while the droplet is up,
the recorded 16 Oct night otherwise (the top bar shows which):
> https://telereason.vercel.app

Code (MIT):
> github.com/BurakSahin00/telereason

Run it yourself (fake mode, no GPU):
> docker compose up --build
> http://localhost:8000

Real mode on your own MI300X:
> infra/droplet.md

Data and credits

- FactoryBench: Hugging Face FactoryBench/FactoryBench, revision 2561c2c9; dataset card: CC BY 4.0. arXiv 2605.07675.

- AURSAD: Leporowski, Tola, Hansen, Iosifidis. Zenodo 2021, DOI 10.5281/zenodo.4487073, CC BY 4.0. We use it as included in FactoryBench.

- voraus-AD (arXiv 2311.04153, CC BY-NC-SA 4.0): cited because the FactoryBench card asks for it. Its items are excluded, so no voraus-AD data appears in any result.

- gpt-oss-120b: openai/gpt-oss-120b, Apache 2.0.

- No datasets or model weights in the repository; they are downloaded at setup time.

- Code: MIT. License notes are informational, not legal advice.

- Team CSAIDev: Hale, Burak, Ömer

Source: DATA_NOTICE.md, README “Data and licenses”